Processingをはじめよう

第8章

動き　その２

目次

- フレームレート
- スピードと方向
- 2点間の移動
- 乱数
- タイマー
- 円運動

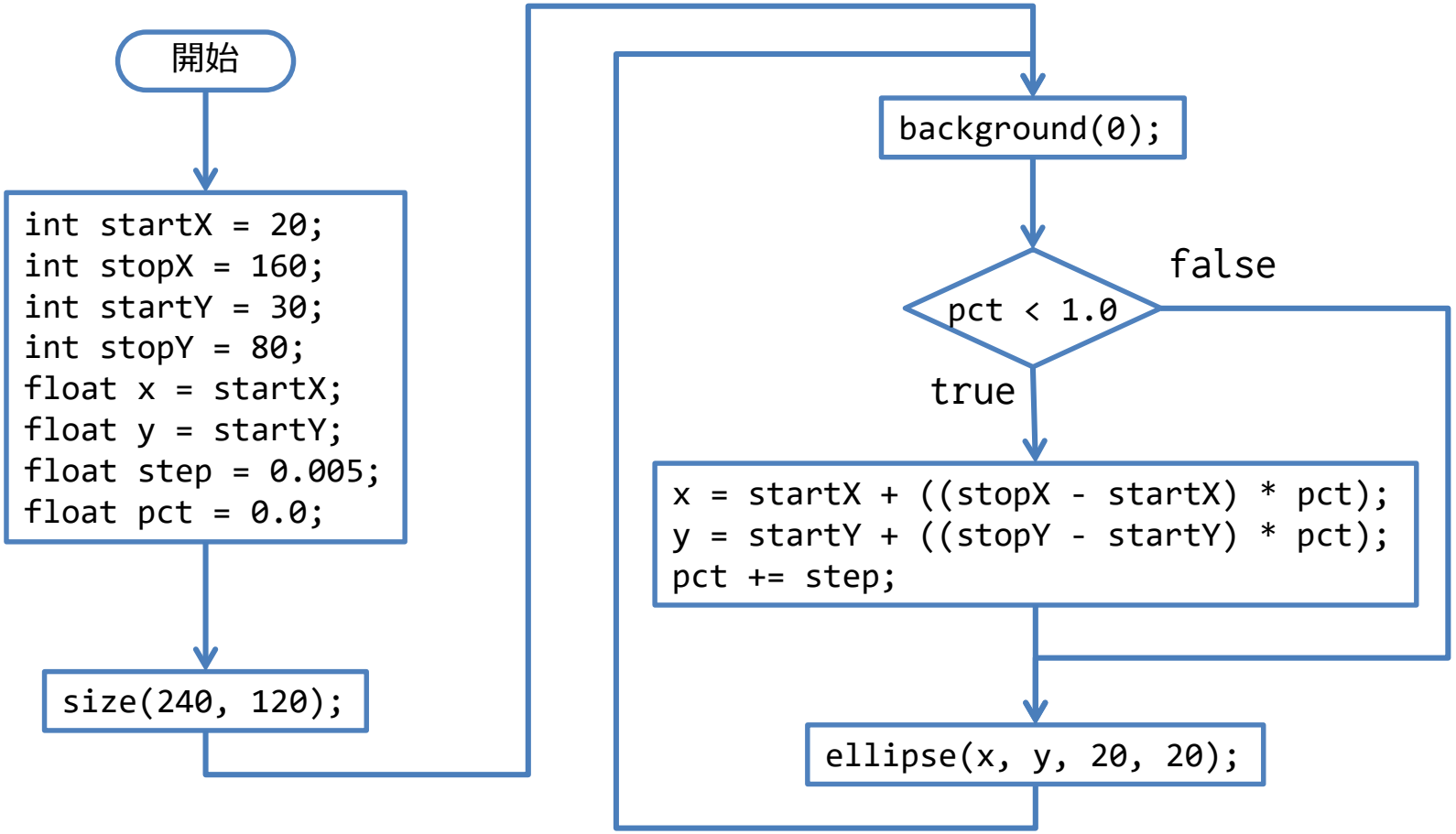
2点間の移動

- Example 8-6 2点間の経路を計算する

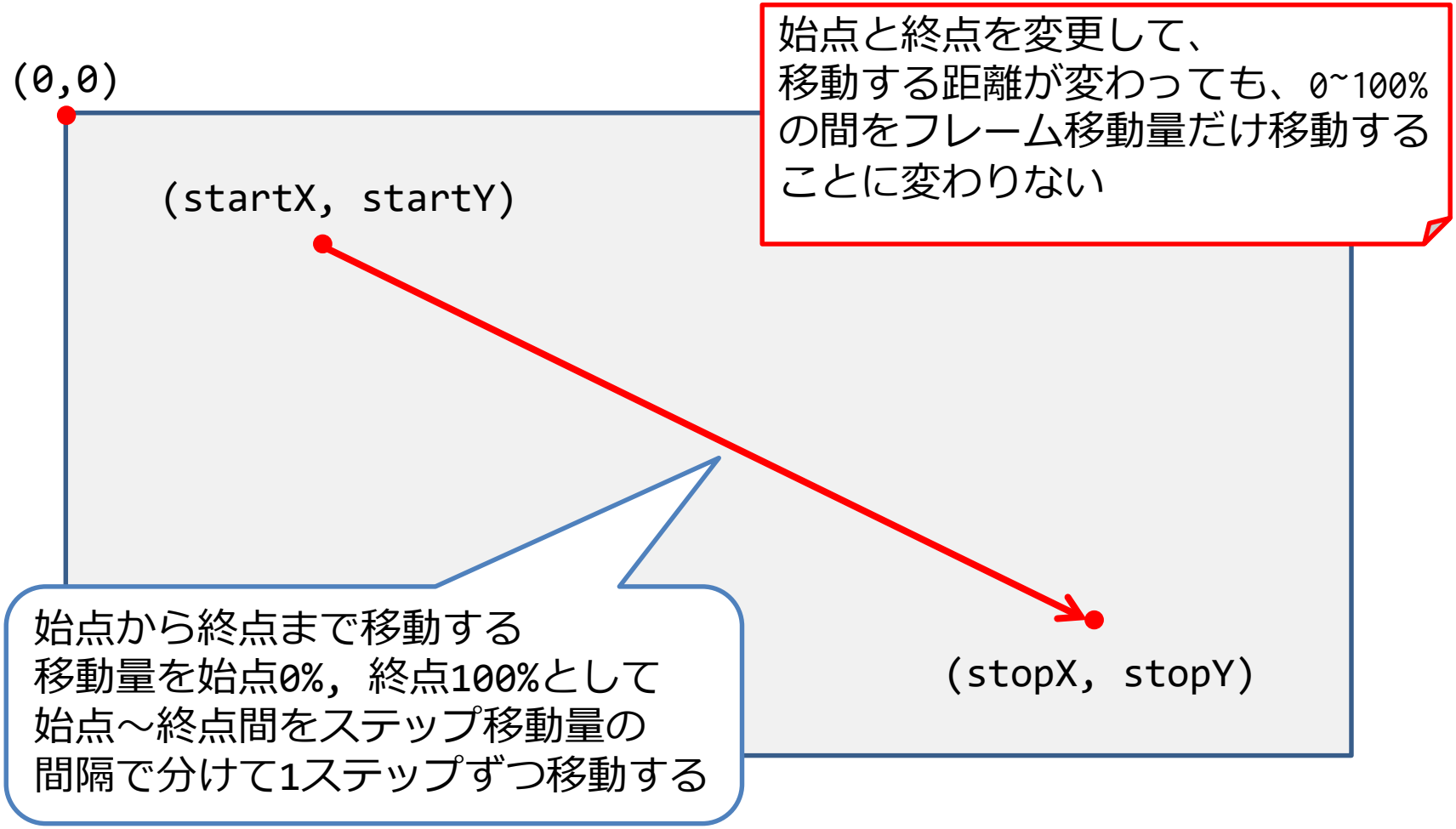
```
int startX = 20;   // 始点のx座標 20
int stopX = 160;   // 終点のx座標 160
int startY = 30;   // 始点のy座標 30
int stopY = 80;    // 終点のy座標 80
float x = startX;  // 現在のx座標 初期値 始点のx座標
float y = startY;  // 現在のy座標 初期値 始点のy座標
float step = 0.005; // 1フレームで移動する割合(0.0 ~ 1.0)
float pct = 0.0;   // 始点から終点まで移動した割合

void setup() {
  size(240, 120);
}

void draw() {
  background(0);
  if (pct < 1.0) { // 移動した割合が1.0より小さい場合
    x = startX + ((stopX - startX) * pct);     // x移動
    y = startY + ((stopY - startY) * pct);     // y移動
    pct += step;                               // 移動した割合 step加算
  }
  ellipse(x, y, 20, 20);
}
```

2点間の移動

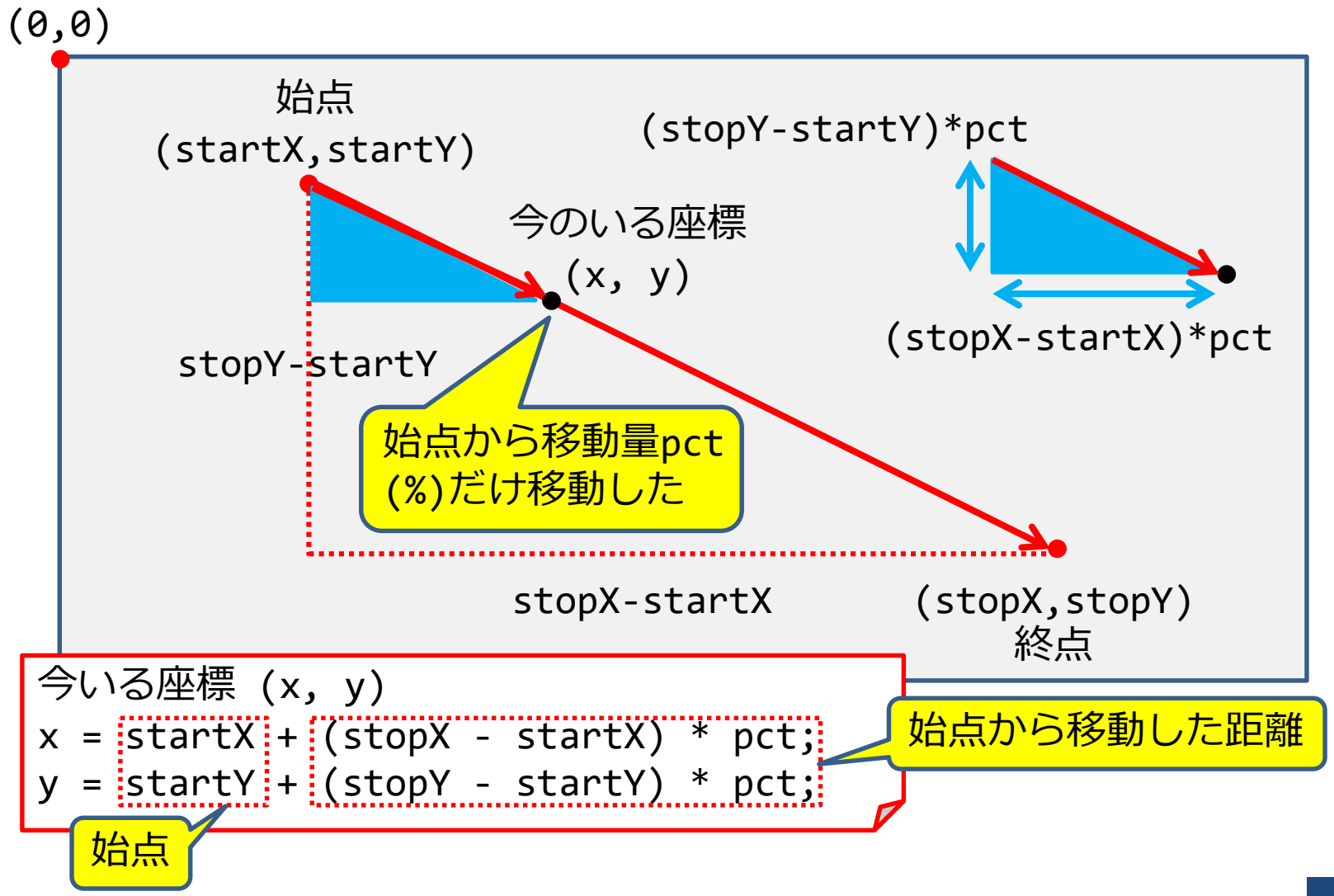
- フローチャート



2点間の移動


(0,0)
(startX, startY)
(stopX, stopY)
始点と終点を変更して、
移動する距離が変わっても、0~100%
の間をフレーム移動量だけ移動する
ことに変わりない
始点から終点まで移動する
移動量を始点0%，終点100%として
始点～終点間をステップ移動量の
間隔で分けて1ステップずつ移動する

2点間の移動距離を計算



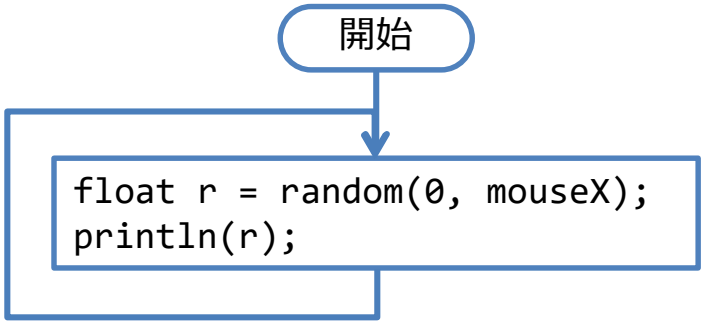
今いる座標 (x, y)

```
x = startX + (stopX - startX) * pct;
y = startY + (stopY - startY) * pct;
```

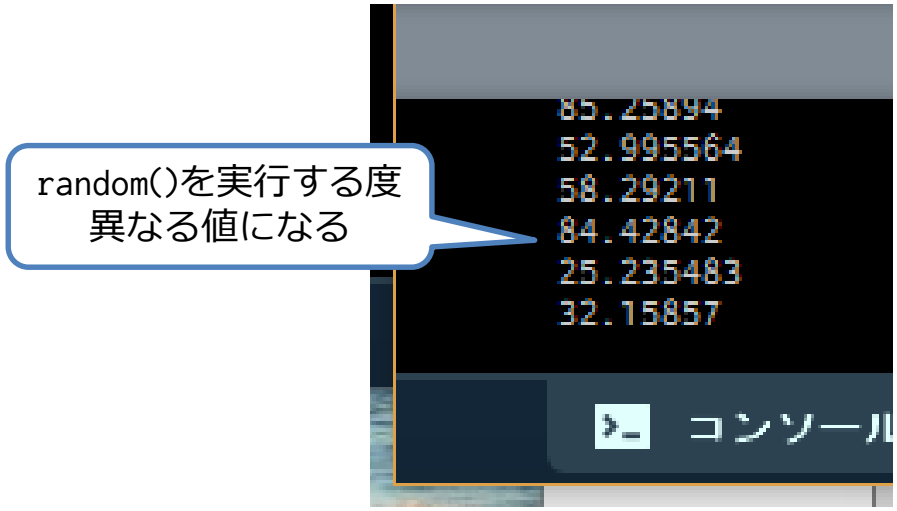
乱数

- Example 8-7 乱数の生成

```
void draw() {
  float r = random(0, mouseX);
  println(r);
}
```



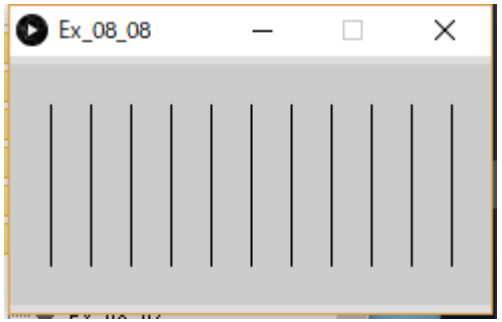
- random()関数
  - 教科書 P232 乱数の生成
    - random(high)
      0以上high未満の乱数生成
    - random(low, high)
      low以上high未満の乱数生成

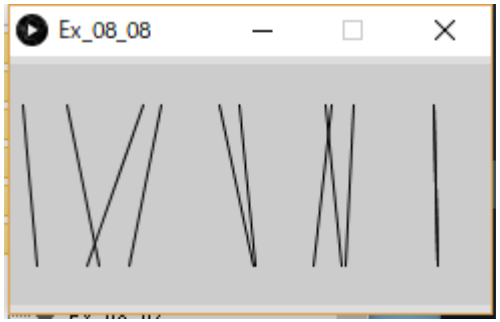


乱数

- Example 8-8 ランダムに描く

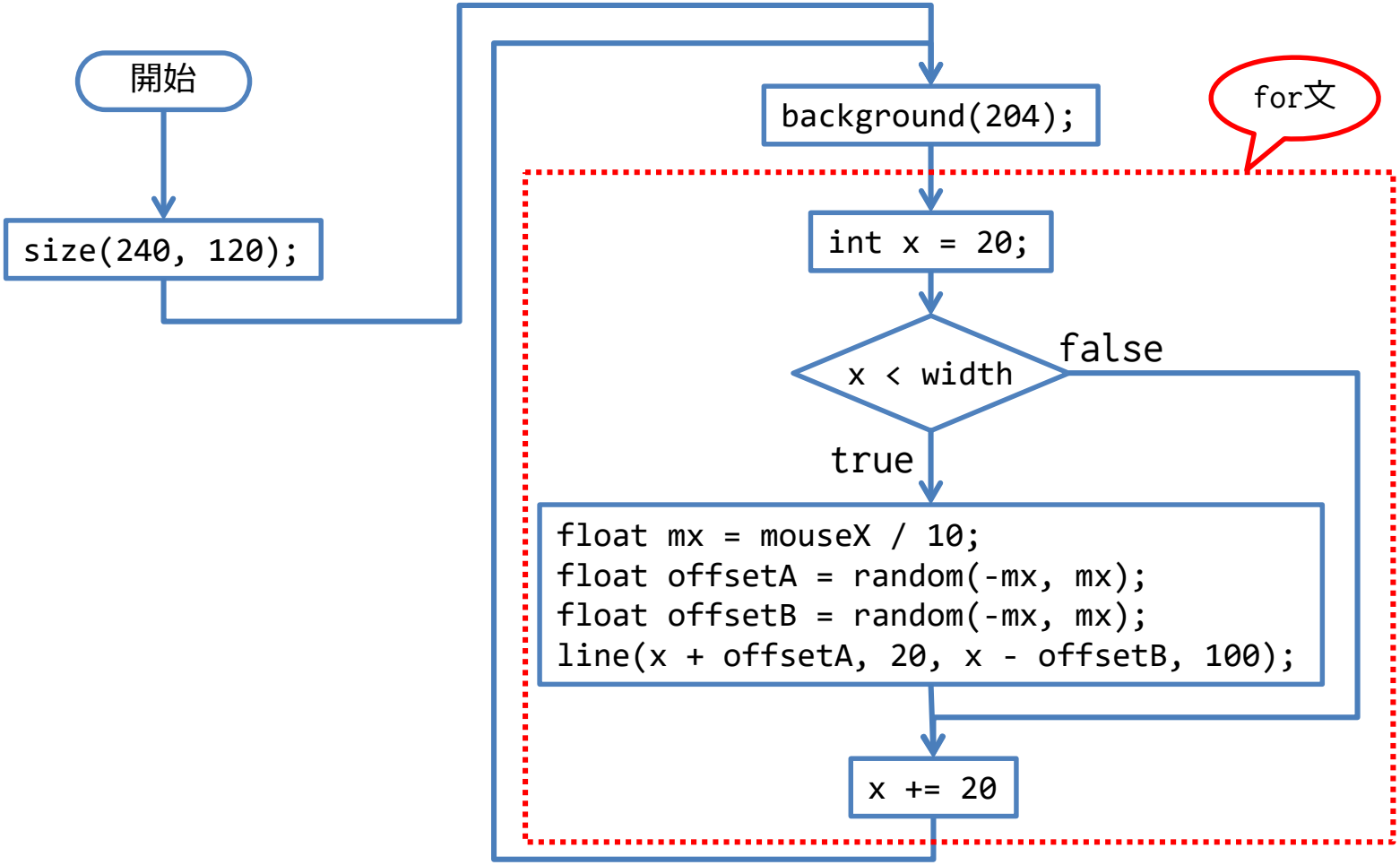
```
void setup() {
  size(240, 120);
}

void draw() {
  background(204);
  for (int x = 20; x < width; x += 20) {
    float mx = mouseX / 10;
    float offsetA = random(-mx, mx);
    float offsetB = random(-mx, mx);
    line(x + offsetA, 20, x - offsetB, 100);
  }
}
```



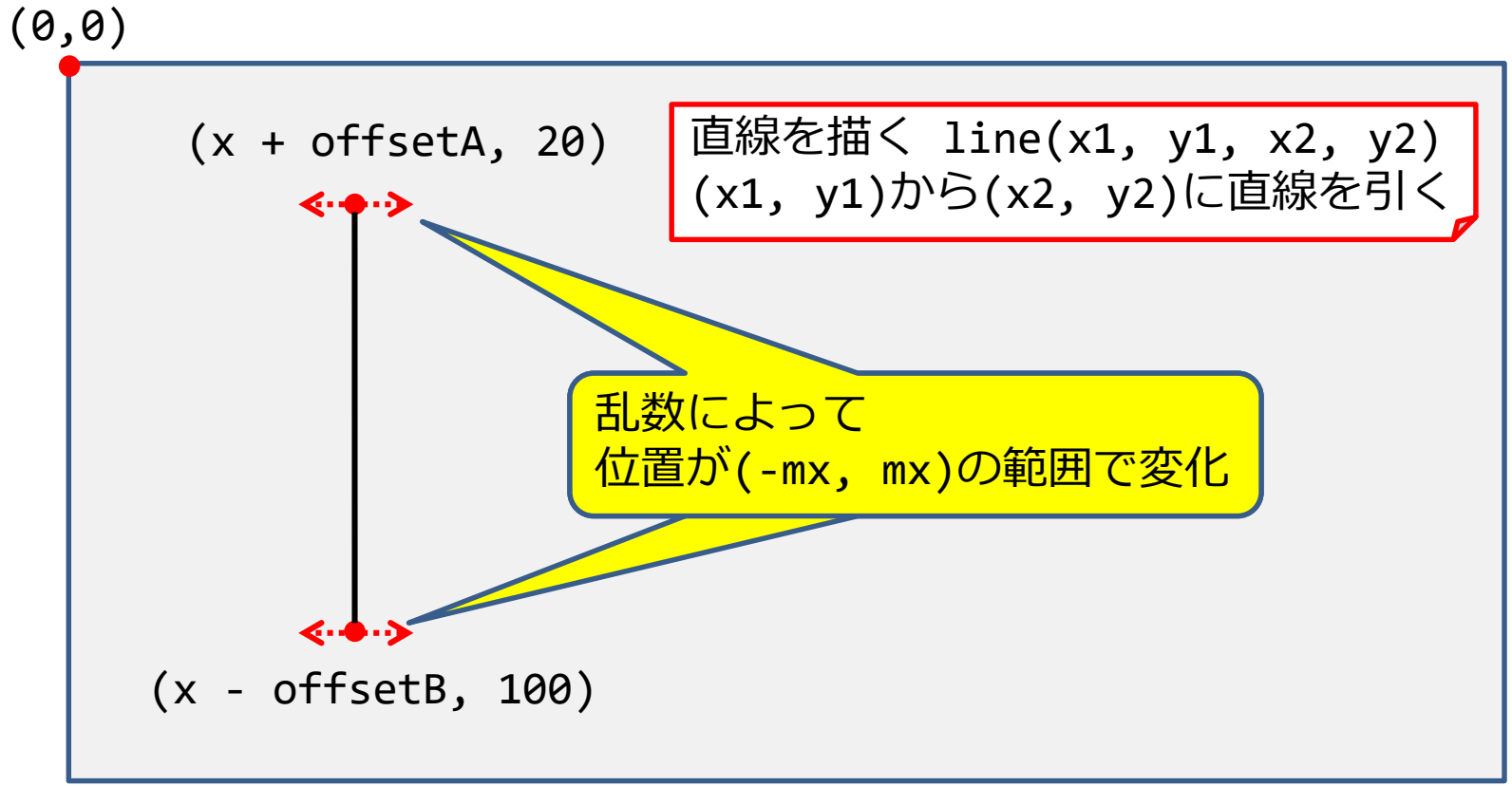


乱数

- フローチャート


開始
size(240, 120);
background(204);
for文
int x = 20;
x < width
false
true
float mx = mouseX / 10;
float offsetA = random(-mx, mx);
float offsetB = random(-mx, mx);
line(x + offsetA, 20, x - offsetB, 100);
x += 20

線

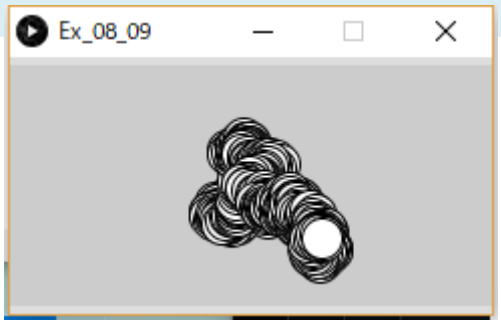

(0,0)
(x + offsetA, 20)
直線を描く line(x1, y1, x2, y2)
(x1, y1)から(x2, y2)に直線を引く
乱数によって
位置が(-mx, mx)の範囲で変化
(x - offsetB, 100)

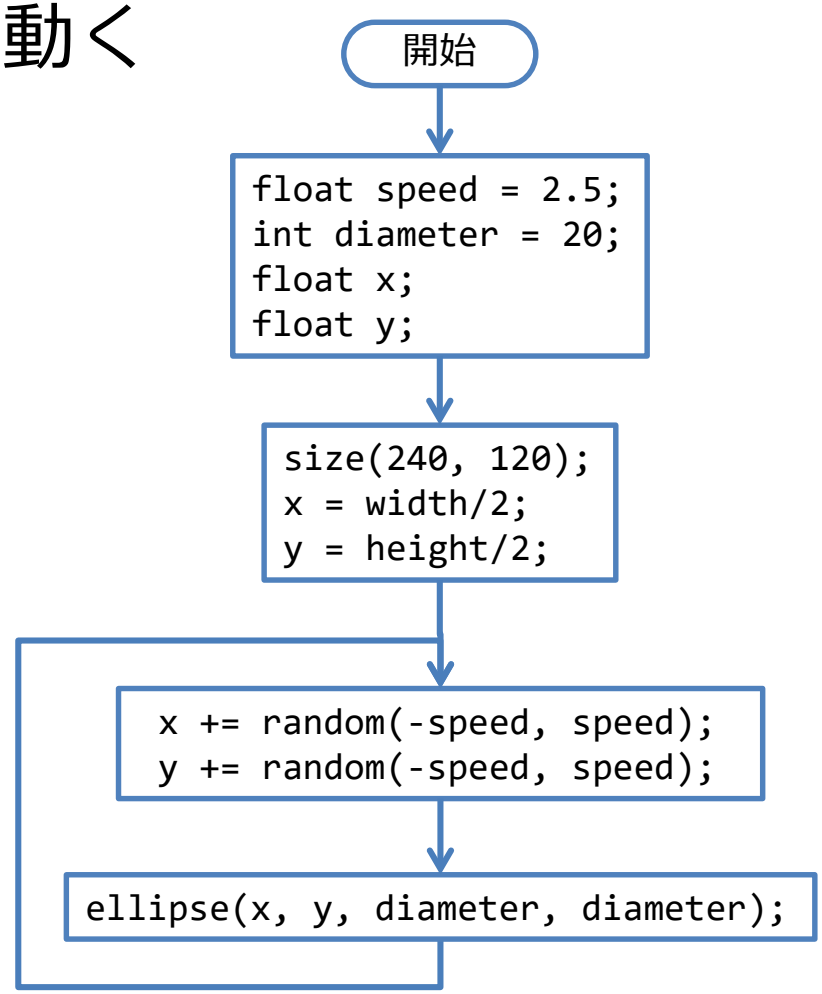
乱数

- Example 8-9 ランダムに動く

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

P120
変更点



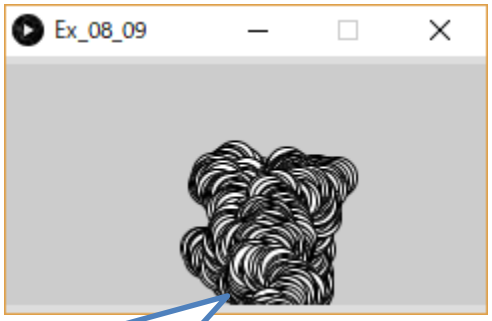


乱数

- P120 変更点
  - 円が画面外に出ないように制限
  - 値を制限する constrain() P230

    constrain(a, min, max)

    aの値をminとmaxの間に制限する



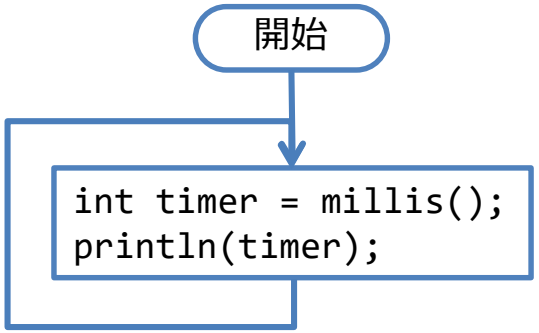
円の中心が画面外に出ない

```
void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  x = constrain(x, 0, width);    //xを0とwidthの間に制限
  y = constrain(y, 0, height);   //yを0とheightの間に制限
  ellipse(x, y, diameter, diameter);
}
```

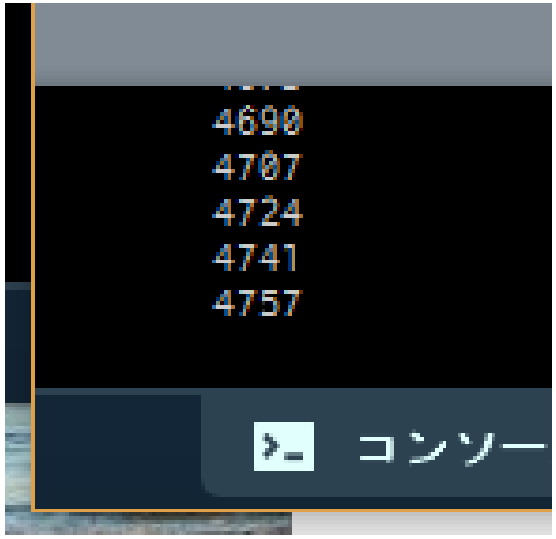
タイマー

- Example 8-10 時間の経過

```
void draw() {
  int timer = millis();
  println(timer);
}
```



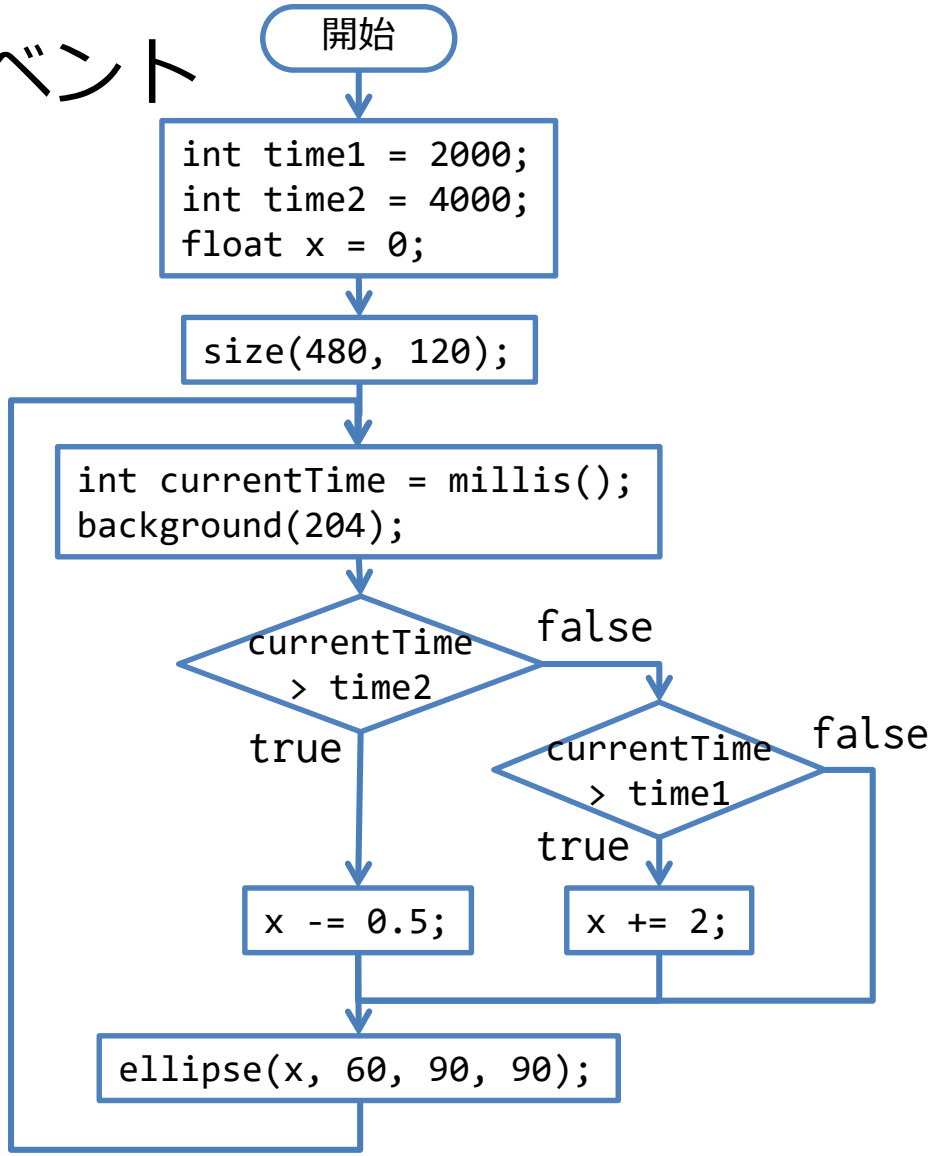
- millis()関数
  実行開始からの
  経過時間（ミリ秒）



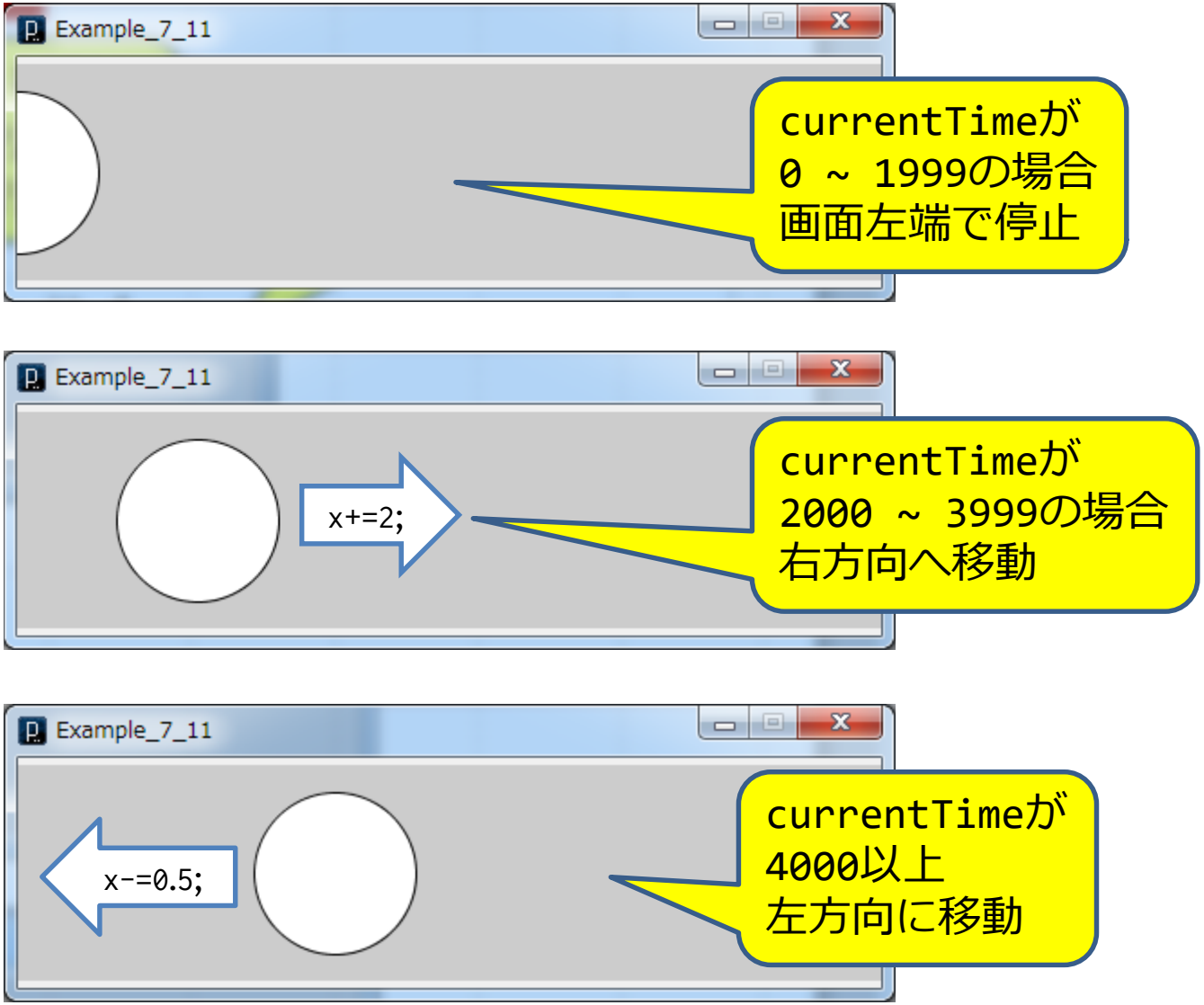
タイマー

- Example 8-11 時限式イベント

```
int time1 = 2000;
int time2 = 4000;
float x = 0;

void setup() {
  size(480, 120);
}

void draw() {
  int currentTime = millis();
  background(204);
  if (currentTime > time2) {
    x -= 0.5;
  } else if (currentTime > time1) {
    x += 2;
  }
  ellipse(x, 60, 90, 90);
}
```



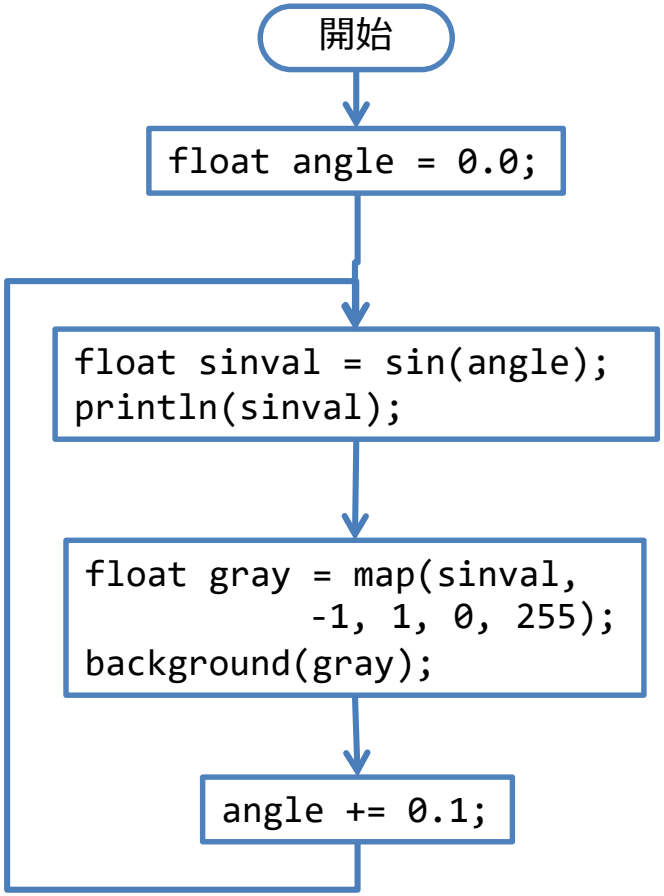
実行結果


Example_7_11
currentTimeが
0 ~ 1999の場合
画面左端で停止
Example_7_11
x+=2;
currentTimeが
2000 ~ 3999の場合
右方向へ移動
Example_7_11
x-=0.5;
currentTimeが
4000以上
左方向に移動

円運動

- Example 8-12 サイン波の値

```
float angle = 0.0;

void draw() {
  float sinval = sin(angle);
  println(sinval);
  float gray = map(sinval, -1, 1, 0, 255);
  background(gray);
  angle += 0.1;
}
```

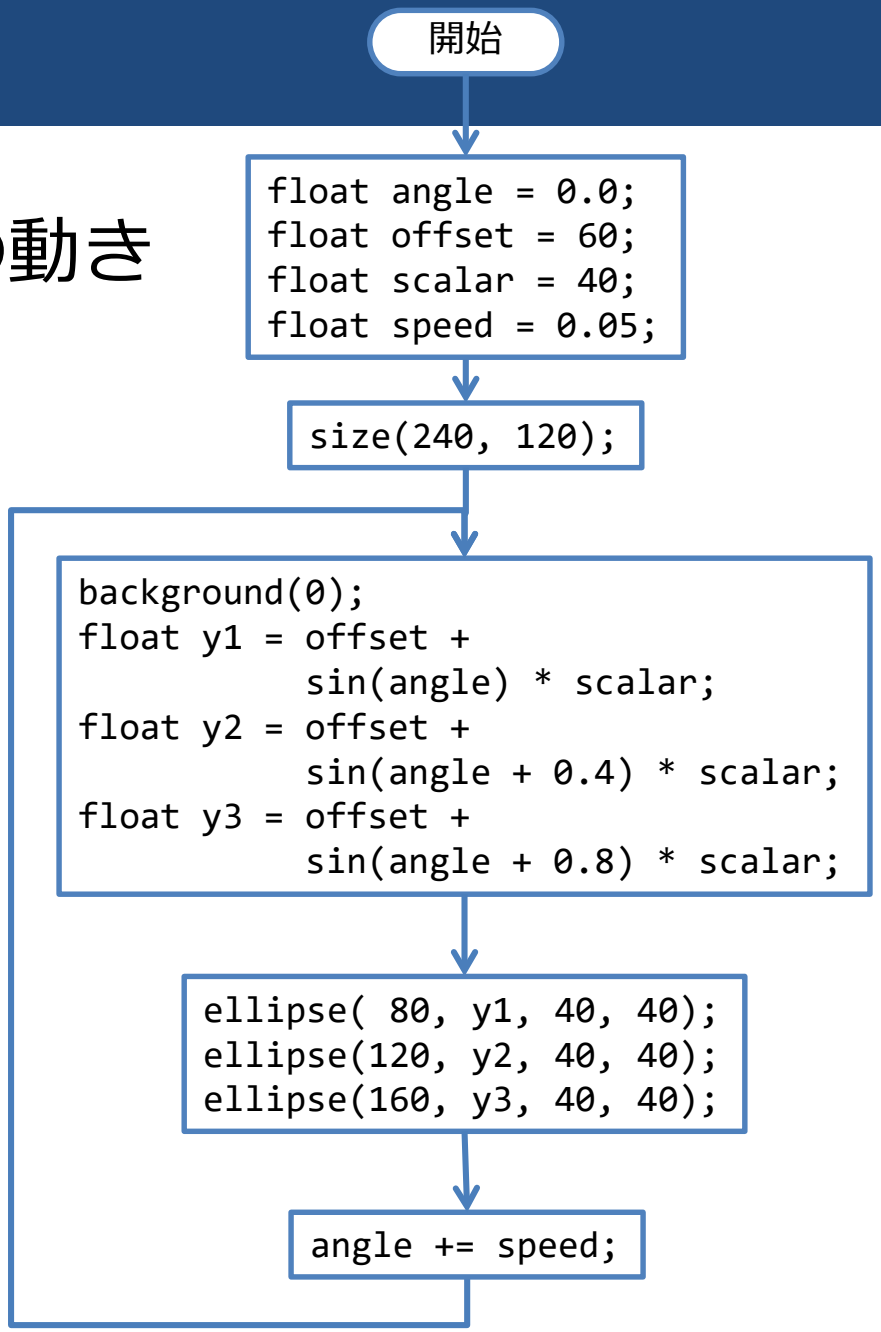


- 教科書 P231
  - sin(a)
    サイン(正弦)を求める (-1.0 ～ +1.0)

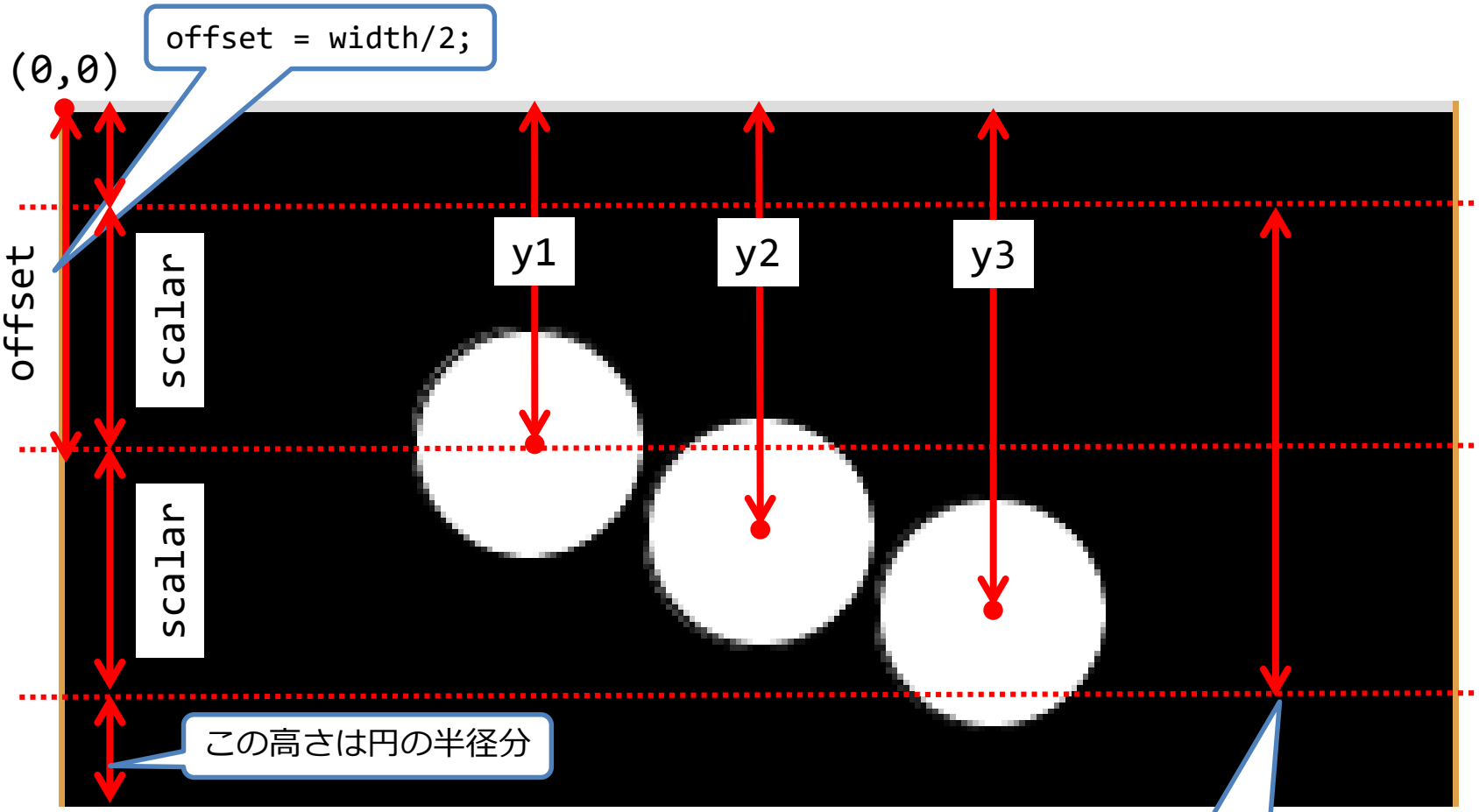
円運動

- Example 8-13 サイン波の動き

```
float angle = 0.0;
float offset = 60;
float scalar = 40;
float speed = 0.05;

void setup() {
  size(240, 120);
}

void draw() {
  background(0);
  float y1 = offset +
             sin(angle) * scalar;
  float y2 = offset +
             sin(angle + 0.4) * scalar;
  float y3 = offset +
             sin(angle + 0.8) * scalar;
  ellipse( 80, y1, 40, 40);
  ellipse(120, y2, 40, 40);
  ellipse(160, y3, 40, 40);
  angle += speed;
}
```



実行結果

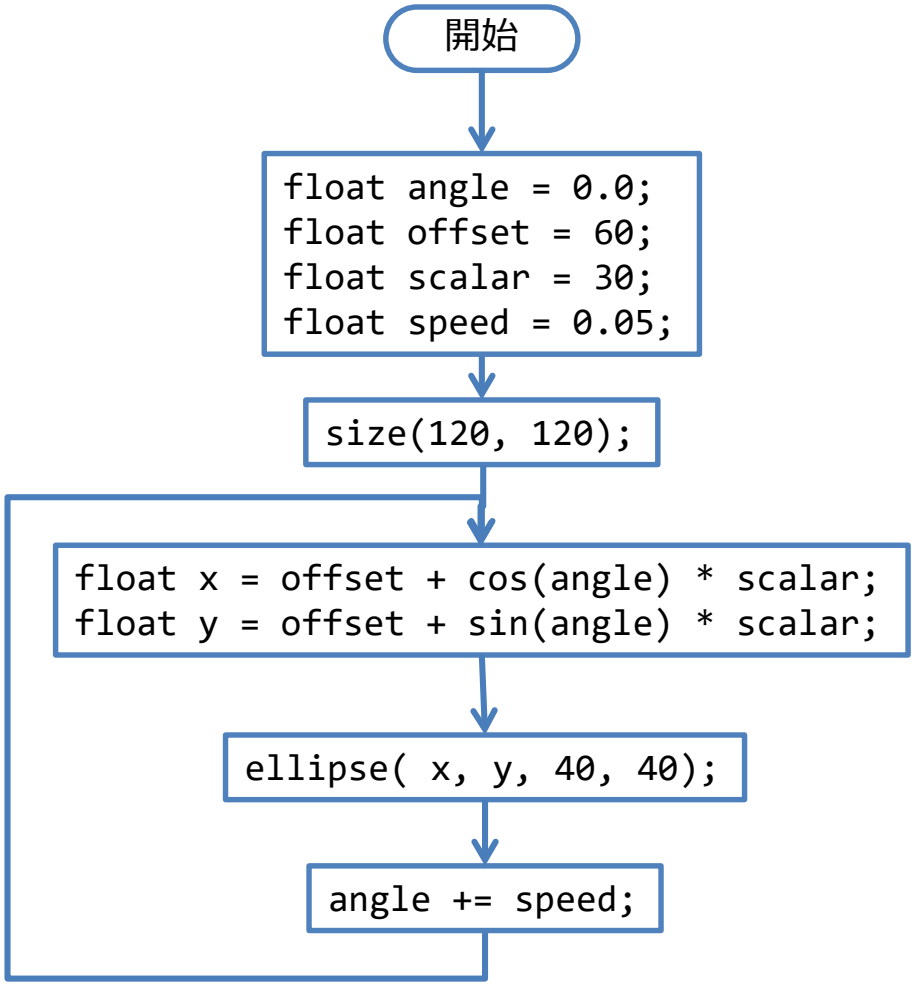



```
y1 = offset + sin(angle) * scalar;
y2 = offset + sin(angle + 0.4) * scalar;
y3 = offset + sin(angle + 0.8) * scalar;
```

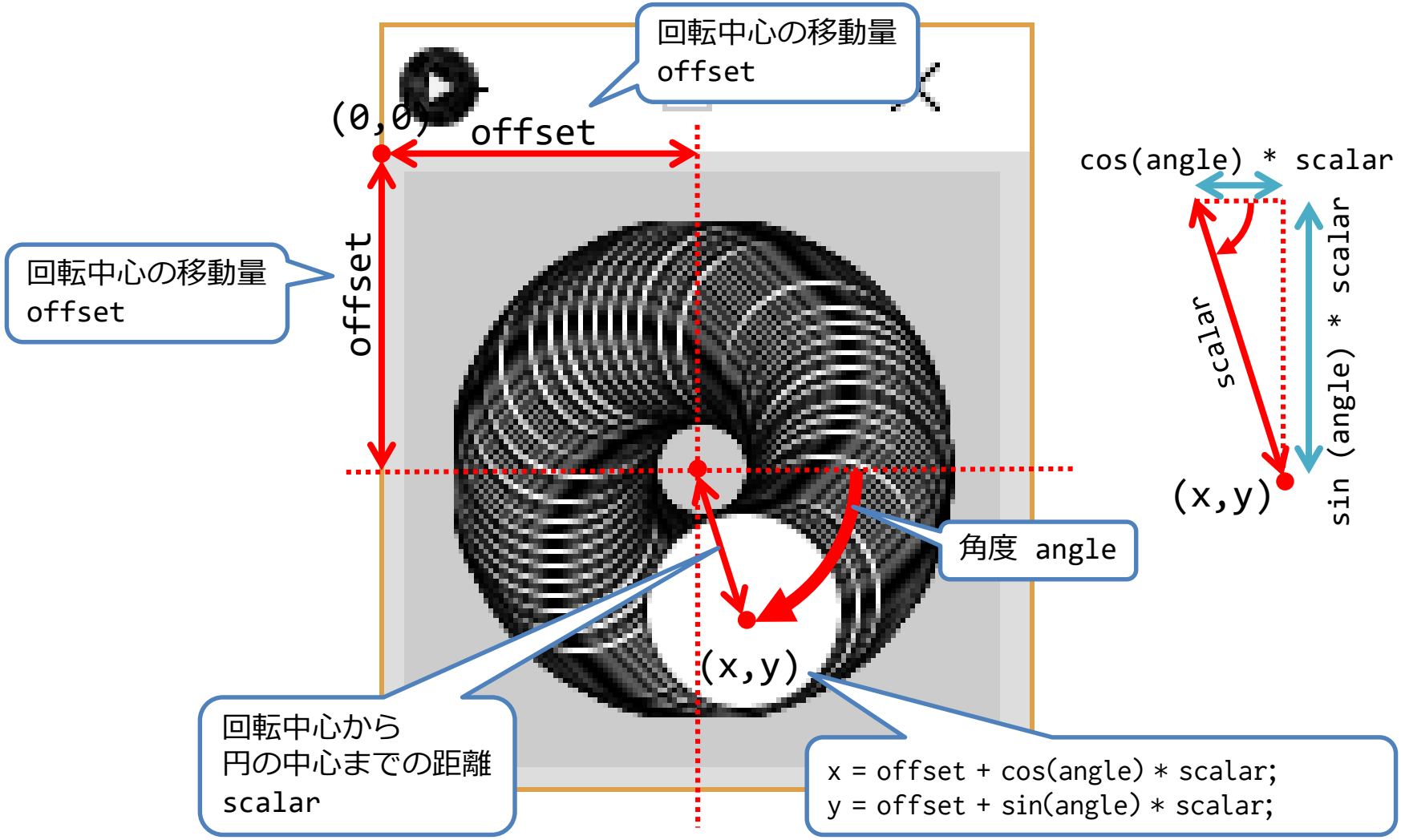
円運動

- Example 8-14 円運動

```
float angle = 0.0;
float offset = 60;
float scalar = 30;
float speed = 0.05;

void setup() {
  size(120, 120);
}

void draw() {
  float x = offset +
            cos(angle) * scalar;
  float y = offset +
            sin(angle) * scalar;
  ellipse( x, y, 40, 40);
  angle += speed;
}
```



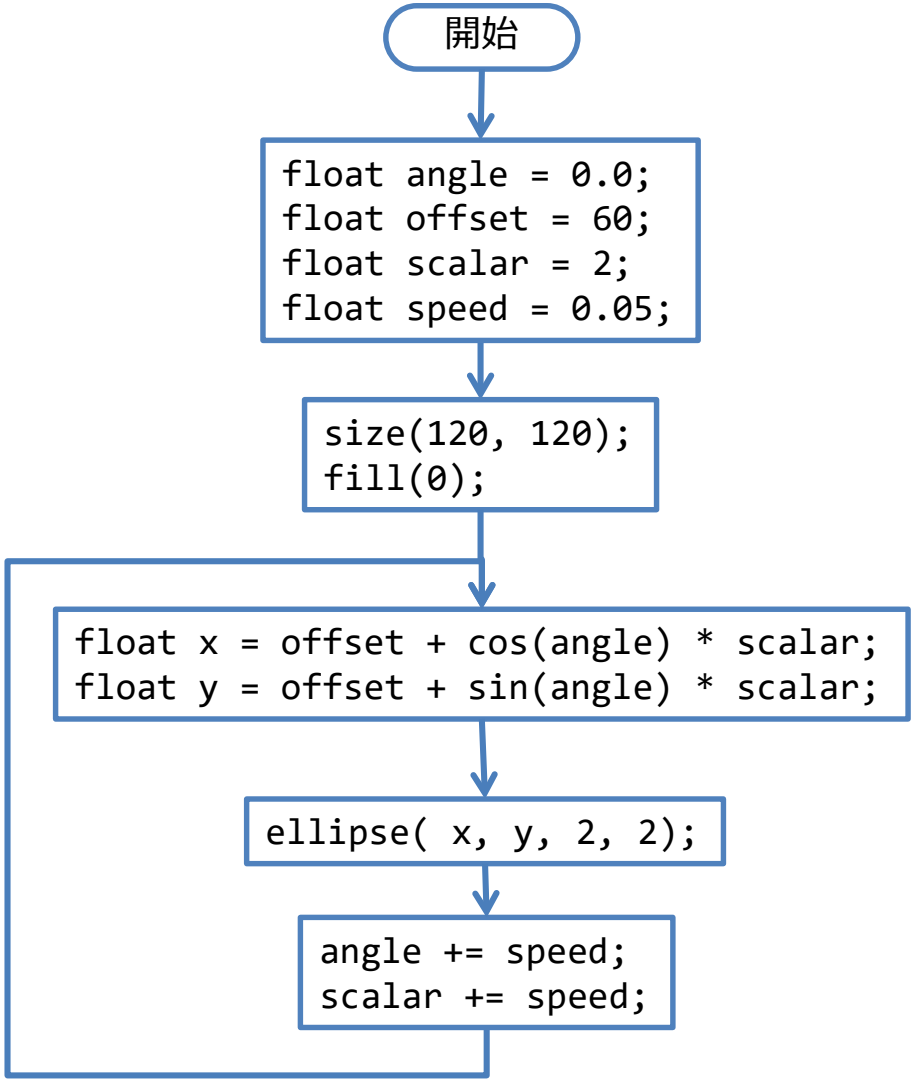
実行結果


回転中心の移動量
offset
(0,0)
offset
回転中心の移動量
offset
offset
cos(angle) * scalar
scalar
sin (angle) * scalar
(x,y)
角度 angle
(x,y)
回転中心から
円の中心までの距離
scalar
x = offset + cos(angle) * scalar;
y = offset + sin(angle) * scalar;

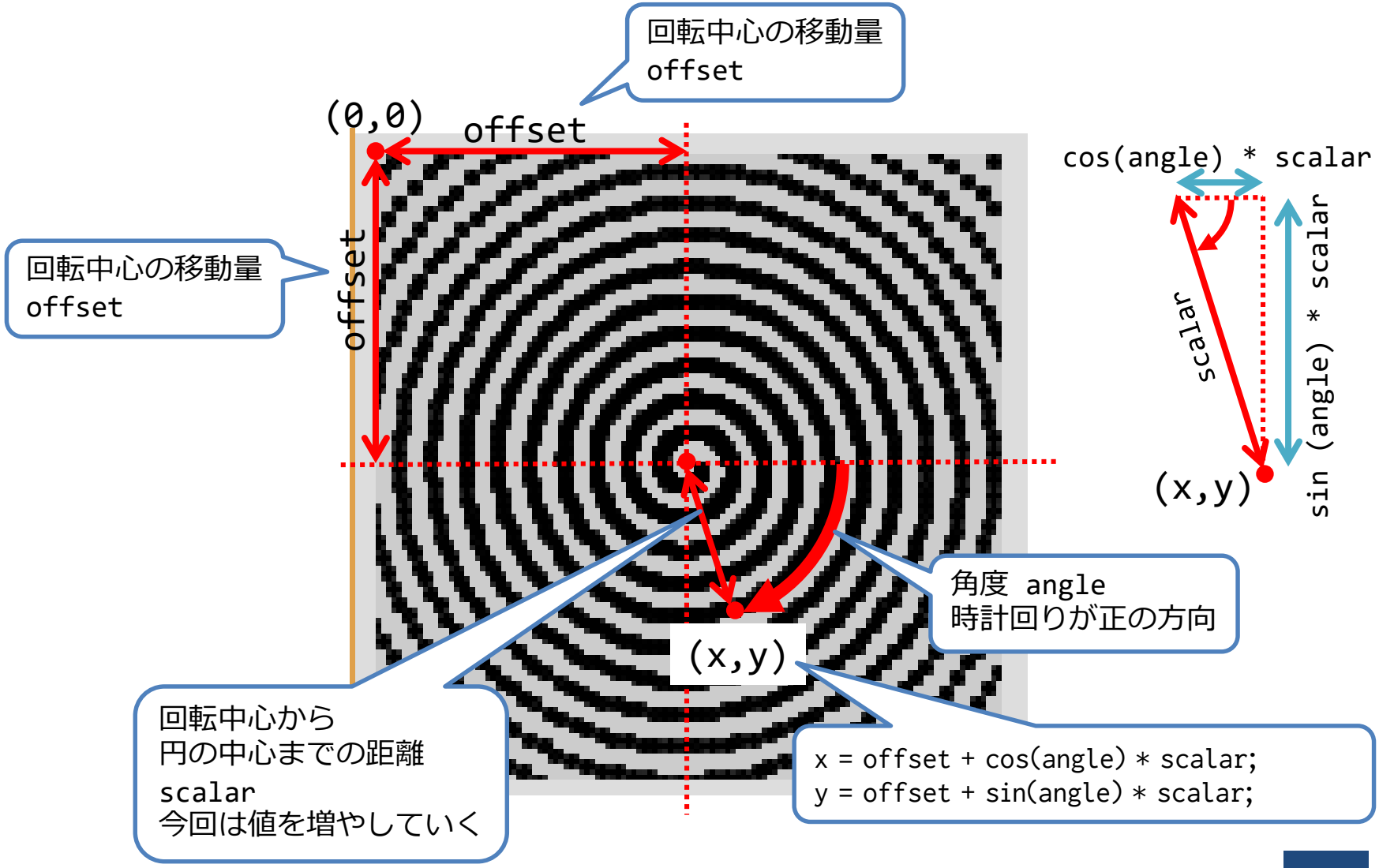
円運動

- Example 8-15 らせん

```
float angle = 0.0;
float offset = 60;
float scalar = 2;
float speed = 0.05;

void setup() {
  size(120, 120);
  fill(0);
}

void draw() {
  float x = offset +
            cos(angle) * scalar;
  float y = offset +
            sin(angle) * scalar;
  ellipse( x, y, 2, 2);
  angle += speed;
  scalar += speed;
}
```



実行結果


回転中心の移動量
offset
(0,0)
offset
回転中心の移動量
offset
offset
cos(angle) * scalar
scalar
sin (angle) * scalar
(x,y)
角度 angle
時計回りが正の方向
(x,y)
回転中心から
円の中心までの距離
scalar
今回は値を増やしていく
x = offset + cos(angle) * scalar;
y = offset + sin(angle) * scalar;